

📖 ДОМЕН 5: БАЗЫ ДАННЫХ — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 5.1: ОСНОВЫ SQL (JUNIOR LEVEL)

5.1.1. Базовые запросы (Junior Level)

№ 5.1.1.1 — SELECT

- **Что нужно знать:** выборка данных, SELECT * vs SELECT column1, column2
- **Вопросы на собеседовании:** "Как выбрать все столбцы из таблицы?"
- **Практика:** Создать таблицу users (id, name, email, age) и выбрать всех пользователей

№ 5.1.1.2 — WHERE

- **Что нужно знать:** фильтрация (=, <>, >, <, >=, <=, LIKE, IN, BETWEEN, IS NULL)
- **Вопросы на собеседовании:** "Как найти всех пользователей старше 18 лет?"
- **Практика:** Найти пользователей с возрастом от 18 до 30 лет

№ 5.1.1.3 — ORDER BY

- **Что нужно знать:** сортировка (ASC, DESC)
- **Вопросы на собеседовании:** "Как отсортировать пользователей по имени?"
- **Практика:** Вывести пользователей в алфавитном порядке

№ 5.1.1.4 — LIMIT / OFFSET

- **Что нужно знать:** пагинация (PostgreSQL: LIMIT 10 OFFSET 20; SQL Server: OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY)
- **Вопросы на собеседовании:** "Как реализовать пагинацию в SQL?"

- **Практика:** Вывести вторую страницу из 10 пользователей

№ 5.1.1.5 — INSERT

- **Что нужно знать:** добавление записей, INSERT INTO ... VALUES (...), INSERT INTO ... SELECT ...
- **Вопросы на собеседовании:** "Как добавить нового пользователя?"
- **Практика:** Добавить 5 пользователей в таблицу

№ 5.1.1.6 — UPDATE

- **Что нужно знать:** обновление записей, UPDATE ... SET ... WHERE ...
- **Вопросы на собеседовании:** "Как изменить email пользователя?"
- **Практика:** Обновить email для пользователя с id = 1

№ 5.1.1.7 — DELETE

- **Что нужно знать:** удаление записей, DELETE FROM ... WHERE ...
- **Вопросы на собеседовании:** "Как удалить пользователя?"
- **Практика:** Удалить пользователя с id = 5

№ 5.1.1.8 — DISTINCT

- **Что нужно знать:** уникальные значения
- **Вопросы на собеседовании:** "Как получить все уникальные города пользователей?"
- **Практика:** Вывести уникальные значения city из таблицы users

№ 5.1.1.9 — Алиасы (AS)

- **Что нужно знать:** переименование столбцов и таблиц
- **Вопросы на собеседовании:** "Зачем нужны алиасы в SQL?"
- **Практика:** Написать запрос с алиасом для COUNT(*)

5.1.2. JOIN и агрегация (Junior → Middle Level)

№ 5.1.2.1 — INNER JOIN

- **Что нужно знать:** только совпадающие строки из обеих таблиц
- **Вопросы на собеседовании:** "Как получить заказы вместе с именами пользователей?"

- **Практика:** Создать таблицу orders (id, user_id, total) и вывести заказы с именами пользователей

№ 5.1.2.2 — LEFT JOIN

- **Что нужно знать:** все строки из левой таблицы, совпадающие из правой
- **Вопросы на собеседовании:** "Как найти пользователей без заказов?"
- **Практика:** Вывести пользователей, у которых нет заказов

№ 5.1.2.3 — RIGHT JOIN

- **Что нужно знать:** все строки из правой таблицы
- **Вопросы на собеседовании:** "Когда использовать RIGHT JOIN?"
- **Практика:** Вывести все заказы, включая пользователей без заказов

№ 5.1.2.4 — FULL OUTER JOIN

- **Что нужно знать:** все строки из обеих таблиц
- **Вопросы на собеседовании:** "В чем разница между LEFT и FULL OUTER JOIN?"
- **Практика:** Объединить пользователей и заказы, показать все записи

№ 5.1.2.5 — GROUP BY

- **Что нужно знать:** группировка, агрегатные функции (COUNT, SUM, AVG, MIN, MAX)
- **Вопросы на собеседовании:** "Как посчитать количество заказов по каждому пользователю?"
- **Практика:** Посчитать общую сумму заказов для каждого пользователя

№ 5.1.2.6 — HAVING

- **Что нужно знать:** фильтрация после GROUP BY (в отличие от WHERE, который работает до)
- **Вопросы на собеседовании:** "В чем разница между WHERE и HAVING?"
- **Практика:** Найти пользователей с суммой заказов > 1000

№ 5.1.2.7 — Подзапросы

- **Что нужно знать:** в SELECT, FROM, WHERE, HAVING

- **Вопросы на собеседовании:** "Как найти пользователя с максимальным возрастом?"
- **Практика:** Найти пользователя с самым высоким возрастом через подзапрос

№ 5.1.2.8 — CTE (Common Table Expression)

- **Что нужно знать:** WITH ... AS для временных таблиц, рекурсивные CTE
- **Вопросы на собеседовании:** "Что такое CTE и когда его использовать?"
- **Практика:** Написать рекурсивный CTE для дерева категорий

№ 5.1.2.9 — Оконные функции

- **Что нужно знать:** ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE()
- **Вопросы на собеседовании:** "Как пронумеровать строки в результате запроса?"
- **Практика:** Вывести топ-3 заказа по сумме для каждого пользователя

№ 5.1.2.10 — Оконные функции (продвинутые)

- **Что нужно знать:** LAG(), LEAD(), SUM() OVER(), AVG() OVER()
- **Вопросы на собеседовании:** "Как вычислить скользящее среднее за 7 дней?"
- **Практика:** Рассчитать скользящее среднее продаж за неделю

5.1.3. Практические задания (Junior → Middle Level)

№ 5.1.3.1

- **Задание:** Создать базу данных интернет-магазина (users, products, orders, order_items) и заполнить тестовыми данными
- **Что проверяет:** Понимание структуры БД

№ 5.1.3.2

- **Задание:** Написать запрос для вывода общей суммы заказов по каждому пользователю с сортировкой по убыванию
- **Что проверяет:** Знание JOIN, GROUP BY, ORDER BY

№ 5.1.3.3

- **Задание:** Написать запрос для вывода топ-5 самых продаваемых товаров
- **Что проверяет:** Знание агрегации и сортировки

№ 5.1.3.4

- **Задание:** Написать рекурсивный запрос для вывода иерархии категорий
- **Что проверяет:** Понимание CTE и рекурсии

РАЗДЕЛ 5.2: НОРМАЛИЗАЦИЯ И ИНДЕКСЫ (JUNIOR → SENIOR)

5.2.1. Нормализация (Middle Level)

№ 5.2.1.1 — 1НФ (Первая нормальная форма)

- **Что нужно знать:** каждое поле атомарно, нет повторяющихся групп
- **Вопросы на собеседовании:** "Что такое 1НФ? Приведи пример нарушения"
- **Практика:** Спроектировать таблицу orders в 1НФ

№ 5.2.1.2 — 2НФ (Вторая нормальная форма)

- **Что нужно знать:** 1НФ + все неключевые поля зависят от всего первичного ключа
- **Вопросы на собеседовании:** "Что такое 2НФ? Как ее достичь?"
- **Практика:** Разбить таблицу order_items на orders и order_items для 2НФ

№ 5.2.1.3 — 3НФ (Третья нормальная форма)

- **Что нужно знать:** 2НФ + нет транзитивных зависимостей
- **Вопросы на собеседовании:** "В чем разница между 2НФ и 3НФ?"
- **Практика:** Убрать транзитивную зависимость city от zip_code в таблице users

№ 5.2.1.4 — BCNF (Нормальная форма Бойса-Кодда)

- **Что нужно знать:** более строгая, чем 3НФ
- **Вопросы на собеседовании:** "Что такое BCNF и когда она нужна?"

- **Практика:** Привести пример таблицы в 3НФ, но не в BCNF

№ 5.2.1.5 — Денормализация

- **Что нужно знать:** когда и зачем (для производительности)
- **Вопросы на собеседовании:** "Когда стоит нарушать нормализацию?"
- **Практика:** Спроектировать схему для аналитического отчета с денормализацией

№ 5.2.1.6 — Суррогатные ключи

- **Что нужно знать:** SERIAL / AUTO_INCREMENT vs натуральные ключи (email, SSN)
- **Вопросы на собеседовании:** "Что лучше: суррогатный или натуральный ключ?"
- **Практика:** Спроектировать таблицу users с суррогатным ключом id и уникальным email

5.2.2. Индексы (Middle → Senior Level)

№ 5.2.2.1 — Кластерный индекс

- **Что нужно знать:** физический порядок данных в таблице (один на таблицу)
- **Вопросы на собеседовании:** "В чем разница между кластерным и некластерным индексом?"
- **Практика:** Создать кластерный индекс на id в PostgreSQL

№ 5.2.2.2 — Некластерный индекс

- **Что нужно знать:** отдельная структура с указателями на данные
- **Вопросы на собеседовании:** "Когда использовать некластерный индекс?"
- **Практика:** Создать некластерный индекс на email в PostgreSQL

№ 5.2.2.3 — B-Tree индекс

- **Что нужно знать:** структура, поиск, вставка, удаление за $O(\log n)$
- **Вопросы на собеседовании:** "Как работает B-Tree индекс под капотом?"
- **Практика:** Проанализировать структуру B-Tree индекса через pg_am

№ 5.2.2.4 — Покрывающий индекс (Covering Index)

- **Что нужно знать:** индекс, который содержит все нужные столбцы
- **Вопросы на собеседовании:** "Что такое покрывающий индекс и зачем он нужен?"
- **Практика:** Создать покрывающий индекс для частого запроса

№ 5.2.2.5 — Частичный индекс (Partial Index)

- **Что нужно знать:** индекс только для подмножества строк
- **Вопросы на собеседовании:** "Когда использовать частичный индекс?"
- **Практика:** Создать индекс только для пользователей с `is_active = true`

№ 5.2.2.6 — Функциональный индекс

- **Что нужно знать:** индекс на выражении (`LOWER(email)`)
- **Вопросы на собеседовании:** "Как сделать поиск без учета регистра быстрым?"
- **Практика:** Создать индекс на `LOWER(email)`

№ 5.2.2.7 — GIN индекс

- **Что нужно знать:** для полнотекстового поиска, JSON, массивов
- **Вопросы на собеседовании:** "Когда использовать GIN индекс?"
- **Практика:** Создать GIN индекс для полнотекстового поиска по `tsvector`

№ 5.2.2.8 — GiST индекс

- **Что нужно знать:** для пространственных данных (PostGIS), геометрии
- **Вопросы на собеседовании:** "Что такое GiST индекс и когда его использовать?"
- **Практика:** Создать GiST индекс для поиска по координатам

№ 5.2.2.9 — BRIN индекс

- **Что нужно знать:** для больших таблиц с коррелированными данными
- **Вопросы на собеседовании:** "Когда использовать BRIN индекс вместо B-Tree?"
- **Практика:** Создать BRIN индекс для таблицы с временными метками

№ 5.2.2.10 — Статистика индексов

- **Что нужно знать:** когда индекс не используется, почему

- **Вопросы на собеседовании:** "Почему запрос не использует индекс?"
 - **Практика:** Проанализировать EXPLAIN и понять, почему используется Seq Scan
-

5.2.3. Практические задания (Middle → Senior Level)

№ 5.2.3.1

- **Задание:** Спроектировать схему БД для социальной сети (пользователи, посты, комментарии, лайки) в 3НФ
- **Что проверяет:** Понимание нормализации

№ 5.2.3.2

- **Задание:** Создать индексы для оптимизации запросов: поиск по имени, поиск по дате, полнотекстовый поиск
- **Что проверяет:** Знание индексов

№ 5.2.3.3

- **Задание:** Оптимизировать медленный запрос с использованием EXPLAIN и индексов
- **Что проверяет:** Навыки оптимизации

№ 5.2.3.4

- **Задание:** Сравнить производительность запросов с индексами и без на 1 млн записей
 - **Что проверяет:** Понимание влияния индексов
-

РАЗДЕЛ 5.3: ОПТИМИЗАЦИЯ ЗАПРОСОВ (JUNIOR → SENIOR)

5.3.1. EXPLAIN (Middle Level)

№ 5.3.1.1 — EXPLAIN

- **Что нужно знать:** как читать план запроса, что такое стоимость (cost)
- **Вопросы на собеседовании:** "Как проанализировать медленный запрос?"
- **Практика:** Выполнить EXPLAIN SELECT * FROM users WHERE age > 30

№ 5.3.1.2 — EXPLAIN (ANALYZE, BUFFERS)

- **Что нужно знать:** реальное выполнение, чтение с диска
- **Вопросы на собеседовании:** "Как узнать, сколько времени выполняется запрос?"
- **Практика:** Выполнить EXPLAIN (ANALYZE, BUFFERS) для сложного запроса

№ 5.3.1.3 — Seq Scan

- **Что нужно знать:** последовательное сканирование, $O(n)$, плохо для больших таблиц
- **Вопросы на собеседовании:** "Почему Seq Scan медленный для больших таблиц?"
- **Практика:** Проанализировать запрос без индекса и найти Seq Scan

№ 5.3.1.4 — Index Scan

- **Что нужно знать:** сканирование по индексу, $O(\log n)$
- **Вопросы на собеседовании:** "В чем разница между Index Scan и Index Only Scan?"
- **Практика:** Создать индекс и выполнить запрос с Index Scan

№ 5.3.1.5 — Bitmap Index Scan

- **Что нужно знать:** сканирование с битовой картой для нескольких индексов
- **Вопросы на собеседовании:** "Как работает Bitmap Index Scan?"
- **Практика:** Найти пользователей по нескольким условиям с Bitmap

№ 5.3.1.6 — Nested Loop Join

- **Что нужно знать:** для маленьких таблиц или с индексами
- **Вопросы на собеседовании:** "Когда используется Nested Loop Join?"
- **Практика:** Анализировать план с Nested Loop

№ 5.3.1.7 — Hash Join

- **Что нужно знать:** для больших таблиц без индексов
- **Вопросы на собеседовании:** "Чем отличается Hash Join от Nested Loop?"
- **Практика:** Сравнить планы запросов с разными JOIN

№ 5.3.1.8 — Merge Join

- **Что нужно знать:** для отсортированных данных
- **Вопросы на собеседовании:** "Когда используется Merge Join?"
- **Практика:** Проанализировать план с Merge Join

№ 5.3.1.9 — Статистика

- **Что нужно знать:** pg_stats, гистограммы, частотность значений
 - **Вопросы на собеседовании:** "Как статистика влияет на план выполнения?"
 - **Практика:** Изучить pg_stats для таблицы users
-

5.3.2. Оптимизация запросов (Senior Level)

№ 5.3.2.1 — Общие правила

- **Что нужно знать:** не использовать SELECT *, использовать EXISTS вместо COUNT(*)
- **Вопросы на собеседовании:** "Как оптимизировать запрос с SELECT *?"
- **Практика:** Переписать запрос с SELECT * на SELECT нужные поля

№ 5.3.2.2 — Индексы

- **Что нужно знать:** добавлять на поля в WHERE, JOIN, ORDER BY, GROUP BY
- **Вопросы на собеседовании:** "Как выбрать поля для индекса?"
- **Практика:** Создать индекс для частого запроса

№ 5.3.2.3 — Избегать функций в WHERE

- **Что нужно знать:** WHERE YEAR(date) = 2024 → WHERE date >= '2024-01-01' AND date < '2025-01-01'
- **Вопросы на собеседовании:** "Почему функции в WHERE делают индексы бесполезными?"
- **Практика:** Переписать запрос с функцией, чтобы использовать индекс

№ 5.3.2.4 — Партиционирование

- **Что нужно знать:** горизонтальное разделение таблиц (by range, by list, by hash)

- **Вопросы на собеседовании:** "Как ускорить запросы к большой таблице с помощью партиционирования?"
- **Практика:** Создать партиционированную таблицу по дате

№ 5.3.2.5 — Временные таблицы

- **Что нужно знать:** для сложных запросов, материализация
- **Вопросы на собеседовании:** "Когда использовать временные таблицы?"
- **Практика:** Оптимизировать сложный запрос с временной таблицей

№ 5.3.2.6 — Подзапросы vs JOIN

- **Что нужно знать:** когда что быстрее
- **Вопросы на собеседовании:** "Что быстрее: подзапрос или JOIN?"
- **Практика:** Переписать подзапрос на JOIN и сравнить производительность

№ 5.3.2.7 — Оконные функции вместо GROUP BY

- **Что нужно знать:** для вычислений без группировки
- **Вопросы на собеседовании:** "Когда оконные функции быстрее GROUP BY?"
- **Практика:** Заменить GROUP BY на оконные функции для сложных расчетов

№ 5.3.2.8 — Оптимизация OR

- **Что нужно знать:** OR часто медленный, лучше использовать UNION ALL или IN
- **Вопросы на собеседовании:** "Как оптимизировать запрос с несколькими OR?"
- **Практика:** Переписать WHERE a=1 OR a=2 на WHERE a IN (1,2)

5.3.3. Практические задания (Middle → Senior Level)

№ 5.3.3.1

- **Задание:** Найти медленный запрос в логах PostgreSQL, проанализировать через EXPLAIN и оптимизировать
- **Что проверяет:** Навыки оптимизации

№ 5.3.3.2

- **Задание:** Создать партиционированную таблицу для логов по месяцам
- **Что проверяет:** Понимание партиционирования

№ 5.3.3.3

- **Задание:** Оптимизировать запрос с 5 JOIN и подзапросами
- **Что проверяет:** Понимание планов выполнения

№ 5.3.3.4

- **Задание:** Настроить pg_stat_statements для анализа медленных запросов
- **Что проверяет:** Знание мониторинга

РАЗДЕЛ 5.4: ВНУТРЕННЕЕ УСТРОЙСТВО СУБД (SENIOR → STAFF)

5.4.1. MVCC и транзакции (Senior Level)

№ 5.4.1.1 — MVCC (Multiversion Concurrency Control)

- **Что нужно знать:** как работает, XMIN, XMAX, видимость строк
- **Вопросы на собеседовании:** "Как PostgreSQL обеспечивает изоляцию транзакций?"
- **Практика:** Посмотреть системные столбцы xmin, xmax в таблице

№ 5.4.1.2 — Уровни изоляции

- **Что нужно знать:** Read Uncommitted, Read Committed, Repeatable Read, Serializable
- **Вопросы на собеседовании:** "В чем разница между Read Committed и Repeatable Read?"
- **Практика:** Продемонстрировать различия на примерах

№ 5.4.1.3 — Serializable Snapshot Isolation (SSI)

- **Что нужно знать:** в PostgreSQL для Serializable
- **Вопросы на собеседовании:** "Как работает Serializable в PostgreSQL?"
- **Практика:** Выполнить конфликтующие транзакции в Serializable режиме

№ 5.4.1.4 — Dirty Read

- **Что нужно знать:** чтение незафиксированных данных
- **Вопросы на собеседовании:** "Что такое Dirty Read?"
- **Практика:** Продемонстрировать в Read Uncommitted

№ 5.4.1.5 — Non-Repeatable Read

- **Что нужно знать:** чтение разных данных в одной транзакции
- **Вопросы на собеседовании:** "Что такое Non-Repeatable Read?"
- **Практика:** Продемонстрировать в Read Committed

№ 5.4.1.6 — Phantom Read

- **Что нужно знать:** появление новых строк в одной транзакции
- **Вопросы на собеседовании:** "Что такое Phantom Read?"
- **Практика:** Продемонстрировать в Repeatable Read

№ 5.4.1.7 — WAL (Write-Ahead Log)

- **Что нужно знать:** журнал для восстановления, репликации
- **Вопросы на собеседовании:** "Как работает WAL в PostgreSQL?"
- **Практика:** Проанализировать WAL-логи

№ 5.4.1.8 — Блокировки

- **Что нужно знать:** row-level, table-level, advisory locks
- **Вопросы на собеседовании:** "Какие блокировки есть в PostgreSQL?"
- **Практика:** Продемонстрировать SELECT FOR UPDATE и дедлок

№ 5.4.1.9 — Deadlock

- **Что нужно знать:** взаимная блокировка, как детектить и предотвращать
- **Вопросы на собеседовании:** "Как PostgreSQL детектирует дедлоки?"
- **Практика:** Создать дедлок и посмотреть на его разрешение

№ 5.4.1.10 — Хранимые процедуры и функции

- **Что нужно знать:** PL/pgSQL, триггеры
- **Вопросы на собеседовании:** "Когда использовать хранимые процедуры?"
- **Практика:** Написать процедуру для обновления статистики

5.4.2. Репликация и шардирование (Staff Level)

№ 5.4.2.1 — Репликация

- **Что нужно знать:** синхронная vs асинхронная, master-slave, streaming replication
- **Вопросы на собеседовании:** "Как настроить репликацию в PostgreSQL?"
- **Практика:** Настроить master-slave репликацию и сделать switchover

№ 5.4.2.2 — Логическая репликация

- **Что нужно знать:** репликация на уровне таблиц, не только всей БД
- **Вопросы на собеседовании:** "Чем логическая репликация отличается от физической?"
- **Практика:** Настроить логическую репликацию для одной таблицы

№ 5.4.2.3 — Шардирование

- **Что нужно знать:** Range (по диапазону), Hash (хеширование), Consistent Hashing (кольцевое)
- **Вопросы на собеседовании:** "Как шардировать данные в распределенной системе?"
- **Практика:** Спроектировать шардирование для 100 млн записей

№ 5.4.2.4 — Consistent Hashing

- **Что нужно знать:** алгоритм кольцевого хеширования, виртуальные ноды
- **Вопросы на собеседовании:** "Как работает Consistent Hashing и зачем он нужен?"
- **Практика:** Реализовать Consistent Hashing на C#

№ 5.4.2.5 — Распределенные транзакции

- **Что нужно знать:** 2PC (Two-Phase Commit)
- **Вопросы на собеседовании:** "Что такое 2PC и в чем его проблемы?"
- **Практика:** Смоделировать 2PC для двух сервисов

№ 5.4.2.6 — Saga Pattern

- **Что нужно знать:** оркестрация vs хореография

- **Вопросы на собеседовании:** "Как реализовать распределенную транзакцию без 2PC?"
- **Практика:** Реализовать Saga для заказа (Order, Payment, Inventory)

№ 5.4.2.7 — CAP-теорема

- **Что нужно знать:** Consistency, Availability, Partition Tolerance
- **Вопросы на собеседовании:** "Что такое CAP-теорема?"
- **Практика:** Выбрать БД для системы с высокой доступностью

№ 5.4.2.8 — PACELC

- **Что нужно знать:** расширение CAP
- **Вопросы на собеседовании:** "В чем отличие PACELC от CAP?"
- **Практика:** Изучить PACELC для разных БД

№ 5.4.2.9 — LSM-дерево

- **Что нужно знать:** Log-Structured Merge Tree (RocksDB, Cassandra)
- **Вопросы на собеседовании:** "Как работает LSM-дерево и чем отличается от B-Tree?"
- **Практика:** Сравнить производительность LSM и B-Tree

№ 5.4.2.10 — Vector Databases

- **Что нужно знать:** Pinecone, Milvus, Weaviate для AI
- **Вопросы на собеседовании:** "Что такое векторная база данных и зачем она нужна?"
- **Практика:** Развернуть Milvus и выполнить поиск по векторам

5.4.3. Практические задания (Senior → Staff Level)

№ 5.4.3.1

- **Задание:** Настроить master-slave репликацию PostgreSQL и сделать switchover
- **Что проверяет:** Знание репликации

№ 5.4.3.2

- **Задание:** Создать дедлок и проанализировать, как PostgreSQL его разрешает

- **Что проверяет:** Знание блокировок

№ 5.4.3.3

- **Задание:** Реализовать Consistent Hashing для распределенного кеша
- **Что проверяет:** Понимание шардирования

№ 5.4.3.4

- **Задание:** Развернуть Vector Database (Milvus) и выполнить поиск по векторам
- **Что проверяет:** Знание современных БД

№ 5.4.3.5

- **Задание:** Реализовать Saga Pattern для распределенной транзакции
- **Что проверяет:** Понимание распределенных систем

РАЗДЕЛ 5.5: NOSQL (MONGO, REDIS, KAFKA) — SENIOR LEVEL

5.5.1. MongoDB (Senior Level)

№ 5.5.1.1 — Документоориентированная БД

- **Что нужно знать:** JSON/BSON, коллекции, гибкая схема
- **Вопросы на собеседовании:** "Когда использовать MongoDB вместо PostgreSQL?"
- **Практика:** Создать коллекцию users и вставить документы

№ 5.5.1.2 — Индексы в MongoDB

- **Что нужно знать:** single-field, compound, text, geospatial
- **Вопросы на собеседовании:** "Как создать индекс в MongoDB?"
- **Практика:** Создать индекс на email и name

№ 5.5.1.3 — Агрегация

- **Что нужно знать:** \$match, \$group, \$project, \$lookup (JOIN)
- **Вопросы на собеседовании:** "Как выполнить группировку в MongoDB?"
- **Практика:** Написать aggregation pipeline для подсчета заказов по пользователям

№ 5.5.1.4 — Шардирование

- **Что нужно знать:** MongoDB sharding, выбор shard key
- **Вопросы на собеседовании:** "Как шардировать MongoDB?"
- **Практика:** Настроить шардирование для коллекции

№ 5.5.1.5 — Репликация

- **Что нужно знать:** replica sets, failover
 - **Вопросы на собеседовании:** "Как MongoDB обеспечивает высокую доступность?"
 - **Практика:** Настроить replica set из 3 узлов
-

5.5.2. Redis (Senior Level)

№ 5.5.2.1 — In-memory БД

- **Что нужно знать:** кеш, сессии, лидерборды
- **Вопросы на собеседовании:** "Когда использовать Redis?"
- **Практика:** Поднять Redis в Docker и выполнить базовые команды

№ 5.5.2.2 — Типы данных

- **Что нужно знать:** String, Hash, List, Set, Sorted Set, Bitmap, HyperLogLog
- **Вопросы на собеседовании:** "Какие типы данных есть в Redis?"
- **Практика:** Использовать Sorted Set для лидерборда

№ 5.5.2.3 — Политики вытеснения

- **Что нужно знать:** LRU, LFU, TTL
- **Вопросы на собеседовании:** "Как Redis управляет памятью?"
- **Практика:** Настроить политику LRU для кеша

№ 5.5.2.4 — Persistent

- **Что нужно знать:** RDB (snapshots) vs AOF (append-only file)
- **Вопросы на собеседовании:** "В чем разница между RDB и AOF?"
- **Практика:** Настроить AOF и выполнить восстановление

№ 5.5.2.5 — Redis Cluster

- **Что нужно знать:** шардирование, репликация

- **Вопросы на собеседовании:** "Как Redis Cluster обеспечивает масштабируемость?"
- **Практика:** Настроить Redis Cluster из 6 узлов

№ 5.5.2.6 — Pub/Sub

- **Что нужно знать:** отправка сообщений между сервисами
- **Вопросы на собеседовании:** "Как использовать Redis для обмена сообщениями?"
- **Практика:** Написать publisher и subscriber на C#

№ 5.5.2.7 — Распределенные блокировки

- **Что нужно знать:** SETNX, Redlock
- **Вопросы на собеседовании:** "Как реализовать распределенную блокировку на Redis?"
- **Практика:** Реализовать распределенную блокировку для критической секции

5.5.3. Apache Kafka (Senior Level)

№ 5.5.3.1 — Kafka

- **Что нужно знать:** архитектура (producers, consumers, topics, partitions, brokers)
- **Вопросы на собеседовании:** "Как устроена архитектура Kafka?"
- **Практика:** Поднять Kafka в Docker и создать топик

№ 5.5.3.2 — Партиционирование

- **Что нужно знать:** выбор partition key, балансировка
- **Вопросы на собеседовании:** "Как выбрать partition key для Kafka?"
- **Практика:** Написать producer с кастомным partitioner

№ 5.5.3.3 — Consumer Groups

- **Что нужно знать:** обработка сообщений несколькими потребителями
- **Вопросы на собеседовании:** "Как Consumer Groups обеспечивают балансировку нагрузки?"
- **Практика:** Создать consumer group из 2 экземпляров

№ 5.5.3.4 — Оффсеты

- **Что нужно знать:** хранение смещений, коммит
- **Вопросы на собеседовании:** "Как Kafka гарантирует Exactly-Once семантику?"
- **Практика:** Настроить `enable.auto.commit=false` и коммитить вручную

№ 5.5.3.5 — ISR (In-Sync Replicas)

- **Что нужно знать:** репликация, гарантии
- **Вопросы на собеседовании:** "Что такое ISR в Kafka?"
- **Практика:** Настроить репликацию и показать, как работает ISR

№ 5.5.3.6 — Kafka Streams

- **Что нужно знать:** обработка потоков, `stateful/stateless`
- **Вопросы на собеседовании:** "Как обрабатывать данные в реальном времени с Kafka?"
- **Практика:** Написать приложение, вычисляющее скользящее среднее за 5 минут

5.5.4. Практические задания (Senior Level)

№ 5.5.4.1

- **Задание:** Использовать Redis как кеш для [ASP.NET](#) Core API (время жизни 5 минут)
- **Что проверяет:** Знание Redis

№ 5.5.4.2

- **Задание:** Написать `producer` и `consumer` для Kafka на C#
- **Что проверяет:** Знание Kafka

№ 5.5.4.3

- **Задание:** Написать MongoDB aggregation pipeline для аналитики заказов
- **Что проверяет:** Знание MongoDB

№ 5.5.4.4

- **Задание:** Реализовать распределенную блокировку на Redis для критической секции
 - **Что проверяет:** Знание распределенных систем
-

🔪 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

SELECT, INSERT, UPDATE, DELETE

WHERE, ORDER BY, LIMIT

INNER JOIN, LEFT JOIN

GROUP BY, COUNT, SUM, AVG

Базовые индексы (кластерные, некластерные)

Middle Level (должен знать 100%)

CTE, оконные функции (ROW_NUMBER, RANK)

Нормализация (1НФ, 2НФ, 3НФ)

Индексы: B-Tree, покрывающие, частичные

EXPLAIN, Seq Scan vs Index Scan

Подзапросы и JOIN

MongoDB: базовые операции, индексы

Redis: базовые типы, TTL

Senior Level (должен знать 100%)

MVCC, уровни изоляции (Read Committed, Repeatable Read, Serializable)

WAL, блокировки, дедлоки

Оптимизация запросов (EXPLAIN ANALYZE, статистика)

Партиционирование, репликация

Шардирование: Consistent Hashing

LSM-дерево vs B-Tree

MongoDB: шардирование, репликация

Redis: персистентность, кластер, Pub/Sub

Kafka: партиционирование, consumer groups, оффсеты

Staff Level (должен знать 100%)

2PC, Saga Pattern

CAP-теорема, PACELC

Vector Databases (Pinecone, Milvus)

Kafka Streams, Exactly-Once

Распределенные транзакции

Внутреннее устройство B-Tree и LSM-деревя

PostgreSQL internals (MVCC, WAL, vacuum)

Настройка производительности: параметры PostgreSQL, мониторинг

ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 5

Задание: Разработать систему аналитики для интернет-магазина.

Шаги:

1. **Схема БД:** спроектировать схему в 3НФ для магазина (users, products, orders, order_items)
2. **Наполнение:** заполнить 1 млн заказов и 10 млн позиций
3. **Оптимизация:** создать индексы для частых запросов
4. **Кеш:** добавить Redis для кеширования популярных товаров
5. **Выгрузка:** добавить Kafka для выгрузки данных в аналитическую систему
6. **Аналитика:** написать запросы для отчетов (топ-10 товаров, выручка по месяцам)
7. **Шардирование:** спроектировать шардирование для 100 млн записей
8. **Репликация:** настроить master-slave репликацию для отказоустойчивости

Проверка:

- Запросы должны выполняться за < 100 мс

- Кеш должен уменьшать время ответа в 10 раз
- Kafka должна обрабатывать 10 000 сообщений/сек